Az `extract_text_from_pdf` Függvény Sorról Sorra Elemzése

Ez a dokumentum a függvény minden egyes sorát külön vizsgálja meg két szempontból:

1. **Szintaktika:** Mit jelentenek a kulcsszavak, zárójelek, adattípusok és könyvtári hívások a Python nyelvben?
2. **Programozói logika:** Mi a célja az adott sornak, hogyan mozog az adat a memóriában, és miért pont így lett megírva?

---

1. Függvényfejléc és Inicializálás

```
def extract_text_from_pdf(pdf_path: Path, ocr_engine: PaddleOCR) -> str:
```

- **Szintaktika:**
  - `def`: A Python kulcsszava új függvény (alprogram) definiálására.
  - `extract_text_from_pdf(...)`: A függvény egyedi azonosítója (neve).
  - `pdf_path: Path`: Első bemenő paraméter. A `: Path` egy típusjelölés (*type hint*), ami jelzi, hogy ez nem sima szöveg (`str`), hanem a `pathlib` modul útvonal-objektuma.
  - `ocr_engine: PaddleOCR`: Második bemenő paraméter. A már korábban inicializált, betöltött PaddleOCR mesterséges intelligencia modellt kapja meg kívülről (hogy ne kelljen minden egyes PDF-nél újra és újra másodpercekig tölteni a modellt a memóriába).
  - `-> str:`: Visszatérési érték típusa. Jelzi a fejlesztőnek és a rendszernek, hogy a függvény lefutása után egyetlen nagy szöveges karakterláncot (`string`) ad vissza.
- **Logika:**
  - A függvény egy adott fizikai fájlból indul ki, átveszi a munkagépet (`ocr_engine`), és a végén szöveget szolgáltat a regex mintakeresőnek.

```
    """Kinyeri a digitális szöveget, ha szkennelt, a PaddleOCR-t hívja meg."""
```

- **Szintaktika:** Háromszoros idézőjelbe zárt dokumentációs blokk (*docstring*).

- **Logika:** A kód olvashatóságát segíti, a fejlesztői környezetek (pl. VS Code) felugró ablakban ezt írják ki, ha az egérrel a függvény fölé viszed a kurzort.

```
full_text = ""
```

- **Szintaktika:** Deklarálunk egy `full_text` nevű változót, és egy üres szöveget (`""`) rendelünk hozzá értékként.
- **Logika:** Ez lesz a gyűjtőtartályunk (*akumulátor*). Ebbe fogjuk oldalanként belefűzni a kinyert szövegeket.

---

2. Digitális Szöveg Kinyerése (1. Fázis)

```
try:
```

- **Szintaktika:** Hibakezelő blokk (*exception handling*) kezdete.
- **Logika:** A program azt mondja: *"Próbáld meg végrehajtani az alábbi sorokat! Ha bármi váratlan hiba történik (pl. sérült a PDF fájl, elfogy a memória), ne állj le azonnal piros hibával, hanem ugorj az* `except` *ágra."*

```
    doc = pymupdf.open(str(pdf_path))
```

- **Szintaktika:**
  - `str(pdf_path)`: A `Path` objektumot szöveggé alakítja (pl. `C:\Users\...\a.pdf`).
  - `pymupdf.open(...)`: Megnyitja a PDF állományt a merevlemezről, és felépít belőle egy manipulálható belső dokumentum-objektumot.
  - `doc = ...`: Ezt a referenciát elmenti a `doc` változóba.
- **Logika:** Ezzel a lépéssel a Python képes lesz lapozni az oldalak között, hozzáférni a belső szövegréteghez vagy képpé alakítani az oldalakat.

```
    for page in doc:
```

- **Szintaktika:** `for ... in ...` ciklus (iteráció).
- **Logika:** Végigmegy a dokumentum minden egyes oldalán a legelsőtől a legutolsóig. Az aktuális oldal referenciája minden körben a `page` változóba kerül.

```
full_text += (
    page.get_text(flags=pymupdf.TEXT_PRESERVE_WHITESPACE) + "\n"
)
```

- **Szintaktika:**
  - `page.get_text(...)`: A PyMuPDF beépített metódusa, ami a PDF belső digitális rétegéből olvassa ki a nyers szöveget.
  - `flags=pymupdf.TEXT_PRESERVE_WHITESPACE`: Speciális beállítás, ami megparancsolja, hogy ne vonja össze a szavakat, hanem pontosan tartsa meg a szóközöket, tabulátorokat és oszlopközöket.
  - `+ "\n"`: Minden oldal végéhez hozzáfűz egy sortörést (*new line*).
  - `+=`: Értéknövelő hozzárendelés. A meglévő `full_text` végéhez ragasztja az új oldal szövegét.
- **Logika:** Ha a PDF nem szkennelt, hanem digitálisan generált (pl. számlázóprogram készítette), akkor itt azonnal, 0.01 másodperc alatt kinyerjük a teljes szöveget, és nem kell hozzá lassú OCR.

---

3. Szkennelt Dokumentum Észlelése (A Döntési Pont)

```
if len(full_text.strip()) < 200:
```

- **Szintaktika:**
  - `full_text.strip()`: Eltávolítja a szöveg elejéről és végéről a láthatatlan karaktereket (szóközök, sortörések, tabulátorok).
  - `len(...)`: Megszámolja, hány darab karakter maradt.
  - `< 200`: Relációs operátor (kisebb, mint 200).
- **Logika (KRITIKUS PONT!):**
  - Itt dönt a gép: *"Találtam elég szöveget digitálisan, vagy ez egy szkennelt kép?"*
  - Ha egy A4-es lapon kevesebb mint 200 karakter van, az szinte biztosan azt jelenti, hogy a szöveg nem digitális betűként van jelen, hanem fényképként (képpontokként). Ekkor belépünk az OCR ágba.

    - **Manuális beavatkozás:** Ha úgy látod, hogy egy lapon van 250 karakter felirat (pl. fejléc), de a lényeg szkennelt kép, ezt a határt megemelheted (pl. `< 400`-ra).

```
print(
    f"  [PaddleOCR] Szkennelt dokumentum észlelve: {pdf_path.name}..."
)
```

- **Szintaktika:** Formázott szövegkiírás (`f-string`). A `{pdf_path.name}` helyére behelyettesíti az éppen futó fájl nevét (pl. `50643367_2026.04.27.pdf`).
- **Logika:** Visszajelzés a felhasználónak a terminálban, hogy tudja, miért lassul le a futás ennél a fájlnál.

---

4. Képkonverzió és Memóriakezelés az OCR Előtt

```
ocr_lines = []
```

- **Szintaktika:** Üres lista inicializálása szögletes zárójellel (`[]`).
- **Logika:** Ebbe a listába fogjuk gyűjteni az OCR által felismert szövegsorokat.

```
for page in doc:
```

- **Szintaktika:** Újabb ciklus a dokumentum oldalain.
- **Logika:** Mivel az OCR kép-alapú, oldalanként kell képpé renderelnünk a dokumentumot.

```
pix = page.get_pixmap(dpi=200)
```

- **Szintaktika:**
    - `page.get_pixmap(...)`: A PyMuPDF funkciója, ami a PDF vektorgrafikus leírását raszteres képpont-mátrixszá (*pixmap*) alakítja át.
    - `dpi=200`: Dots Per Inch (felbontás).
- **Logika:**
    - A 200 DPI a kompromisszumos érték a sebesség és az olvashatóság között.
    - **Manuális beavatkozás:** Ha nagyon apró vagy elmosódott kézírásos számok vannak a CMR-en, megemelheted `dpi=300`-ra. Tisztább lesz a kép az OCR-nek, de kicsit több

memóriát és időt igényel.

```
img_bytes = pix.tobytes("png")
```

- **Szintaktika:** A pixmap képadatait PNG formátumú nyers bájtsorozattá alakítja a memóriában.
- **Logika:** Ahelyett, hogy kiírnánk egy ideiglenes `temp.png` fájlt a lassú merevlemezre/SSD-re, a RAM-ban tartjuk az adatot.

```
img_np = np.frombuffer(img_bytes, dtype=np.uint8)
```

- **Szintaktika:**
  - `np.frombuffer(...)`: A NumPy könyvtár metódusa.
  - `dtype=np.uint8`: 8 bites előjel nélküli egész számként (0–255 közötti értékek) értelmezi a bájtokat (ez a képpontok szabványos formátuma).
- **Logika:** A nyers bájtokból egy dimenzionális matematikai tömböt hozunk létre a memóriában.

```
img = cv2.imdecode(img_np, cv2.IMREAD_COLOR)
```

- **Szintaktika:**
  - `cv2.imdecode(...)`: Az OpenCV könyvtár függvénye, ami a tömbből kibontja a PNG képet egy 3 csatornás (színes) képmátrixszá.
- **Logika:** A PaddleOCR nem tud közvetlenül PDF-et olvasni; neki egy ilyen OpenCV képmátrixra (`ndarray`) van szüksége.

---

5. A PaddleOCR Motor Meghívása és Szövegösszefűzés

```
result = ocr_engine.ocr(img, cls=True)
```

- **Szintaktika:**
  - `ocr_engine.ocr(...)`: Meghívja a PaddleOCR neurális hálóját a megadott `img` képre.
  - `cls=True`: Bekapcsolja a beépített szög-osztályozót (*angle classifier*).
- **Logika (KRITIKUS PONT!):**

- Ez az a sor, ahol a mélytanulási modell dolgozik: felismeri a szövegdobozokat, megállapítja a dőlésszöget (ha fejjel lefelé van a lap, automatikusan megfordítja 180 fokkal a memóriában), és kiolvassa a karaktereket.
- A `result` egy többszörösen beágyazott lista lesz, amely koordinátákat, szöveget és megbízhatósági értéket (*confidence score*) tartalmaz.

```
if result and result[0]:
```

- **Szintaktika:** Logikai feltétel vizsgálata (`and`).
- **Logika:** Biztonsági ellenőrzés. Ha a kép teljesen fehér vagy üres volt, a PaddleOCR üres listát ad vissza. Ez a feltétel megvédi a kódot attól, hogy üres listára hivatkozva összeomoljon (`IndexError`).

```
    for line in result[0]:
      text_line = line[1][0]
      ocr_lines.append(text_line)
```

- **Szintaktika:**
  - `for line in result[0]:`: Végigmegy a képen megtalált összes szövegdobozon.
  - `line[1][0]`: A PaddleOCR adatstruktúrája így néz ki: `[ [koordináták], (szöveg, pontosság) ]`.
    - `line[1]` kiválasztja a `(szöveg, pontosság)` párt.
    - `line[1][0]` ebből kiemeli magát a felismert szöveget (pl. `"Loadinglist: HU03_DE2604270001"`).
  - `ocr_lines.append(text_line)`: Hozzáfűzi a megtalált sort az `ocr_lines` listánkhoz.

```
return "\n".join(ocr_lines)
```

- **Szintaktika:** A `"\n".join(...)` metódus a lista összes szöveges elemét egyetlen nagy szöveggé fűzi össze, közéjük sortörést illesztve. A `return` kilépteti a függvényt és visszaadja ezt a szöveget.
- **Logika:** Ezzel befejeződik a szkennelt ág futása; a kinyert OCR szöveg visszakerül a hívóhoz (`main`).

---

6. Hibaelhárítás és Visszatérés

```
except Exception as e:
  print(f"  [Hiba] PDF feldolgozása sikertelen: {e}")
```

- **Szintaktika:**
  - `except Exception as e:`: Elkapja a `try` blokkban fellépő bármilyen hibát, és elmenti azt az e változóba.
  - Kiírja a képernyőre a hiba pontos technikai okát.
- **Logika:** Megakadályozza, hogy egyetlen sérült vagy jelszóval védett PDF miatt leálljon az egész 100 darabos köteg feldolgozása.

```
return full_text
```

- **Szintaktika:** A függvény végső visszatérése.
- **Logika:** Ha a PDF nem volt szkennelt (több mint 200 karakter volt benne digitálisan), akkor a kód nem lépett be az `if` blokkba, így közvetlenül ide ugrik, és visszaadja az 1. fázisban kinyert tiszta digitális szöveget.

---

7. Hol érdemes manuálisan beavatkoznod a jövőben?

Sor / Beállítás	Mi a szerepe?	Mikor változtasd meg?
`if len(full_text.strip()) < 200:`	Határvonal a digitális és szkennelt PDF között.	Emeld meg (pl. 350-re), ha a fejléc miatt digitálisnak hiszi a szkennelt lapot.
`pix = page.get_pixmap(dpi=200)`	Képfelbontás minősége az OCR előtt.	Állítsd `dpi=300`-ra, ha rossz minőségű, apró számsorokat nem olvas el.
`result = ocr_engine.ocr(img, cls=True)`	Dőlés- és szögfelismerés.	Ha biztosan egyenes minden lap, a `cls=False` beállítással kb. 25%-kal gyorsítható a futás.